GenSpark

SDLC Foundations: Requirements Gathering and System Design Basics

Quick Recap

In Week 1, you covered the six phases of the SDLC (requirements, design, implementation, testing, deployment, maintenance) and how responsibilities shift across them. This week you move from concept to practice. This article takes a closer look at the two phases that happen before a single line of code gets written: requirements gathering and system design. You will practice both on the Tic-Tac-Toe project this week.

Why These Phases Come First

Writing code is the expensive part of a project, and fixing a misunderstanding after the code exists is more expensive than fixing it on paper. A wrong assumption in a requirements list costs you a conversation. The same assumption found after you have built, tested, and shown the system costs you rewrites, and sometimes a client's trust. Skipping or rushing these phases is one of the most common causes of failed or reworked projects. The code is not necessarily bad. It solved the wrong problem.

You have already felt a small version of this. In the testing articles, deciding the expected result before running a test kept you honest. Requirements and design do the same thing for a whole project.

Requirements Gathering

Requirements gathering is the process of figuring out exactly what a system needs to do, and under what limits, before you build it.

Types of requirements

Type	What it describes	Tic-Tac-Toe example
Functional	What the system must *do*	"The game detects when a player has won."
Non-functional	How well it must do it: performance, security, usability, maintainability	"The Hard AI chooses a move in a few seconds on a 5 by 5 board."
Constraints	Limits you must work within	"Runs in the terminal on Python 3.10 or later."
Business rules	Rules of the domain that the system must follow	"A win needs 3 marks in a row on a 3 by 3 board and 4 on larger boards."

Functional requirements are the ones people think of first. Non-functional requirements and constraints are the ones people forget, and they often decide whether a system is actually acceptable. A banking app that does all the right things but takes a minute to load a balance has met every functional requirement and still failed.

Who is the requirement for?

Every system has **stakeholders**, the people who use it, pay for it, run it, or are affected by it. A requirement only makes sense in relation to someone. Ask:

- Who will use this, and what problem does it solve for them?
- Who pays for it or approves it?
- Who has to run or maintain it afterward?

These are often different people with different needs. On a client engagement, the person who describes the problem is not always the person who will use the result.

Asking good questions

Clients rarely describe their needs in precise, technical terms. They describe a goal. Your job is to turn it into something you can build and verify. Take a request like this:

> *"I just want to see where my money's going each month."*

That sounds simple, but it hides a dozen decisions. Good clarifying questions turn it into something concrete:

- *Where does the data come from?* Typed in by hand, a bank export, a spreadsheet?
- *What does "money going" mean?* Categories, merchants, or accounts?
- *What does "see" mean?* A table, a chart, an email at the end of the month?
- *What counts as "each month"?* Calendar months, or pay periods?
- *How many accounts and how much history?*
- *Who else will look at it?* Just you, or a partner or an accountant too?
- *What would make you say "this works"?*

Useful habits:

- **Ask for examples.** "Walk me through the last time you tried to do this."
- **Restate what you heard.** "So you want a monthly total per category, built from the bank export. Is that right?"
- **Show something early.** A sketch or a rough prototype brings out requirements people did not know they had.
- **Listen for hidden constraints.** "It has to work on my old laptop" is a requirement.
- **Write everything down.** A requirement that exists only in someone's memory will change over time.

Writing testable requirements

A good requirement is a clear statement that you could later verify as done or not done. Vague words are the warning sign:

Weak (vague)	Stronger (testable)
"The game should be easy to use."	"An invalid move shows an error message and lets the

	same player try again."
"The AI should be smart."	"The Hard AI never loses a game on a 3 by 3 board."
"It should be fast."	"The Hard AI chooses a move in no more than 3 seconds on a 5 by 5 board."
"Handle bad input."	"Letters, a blank entry, and numbers outside the board are rejected without using the player's turn."
"Support different board sizes."	"The player can choose a 3 by 3, 4 by 4, or 5 by 5 board before the game starts."

Watch for words like *fast*, *easy*, *user-friendly*, *robust*, *flexible*, *appropriate*, *etc.*, and *and so on*. Each one hides a question. Replace it with a number, an example, or a list.

Acceptance criteria

An **acceptance criterion** is a specific check that says a requirement is met. A handy format is *Given, When, Then*:

```
Given a 3 by 3 board with an X in the top left corner,
When player O tries to place a mark in the top left corner,
Then the move is rejected and it is still O's turn.
```

Look at how close this is to a test. You can turn it directly into a pytest function (arrange the board, act with `place_mark`, assert the result), or into a row in a manual test plan. That is the point. Requirements, acceptance criteria, and tests are the same information at different levels of detail.

User stories

Many teams capture requirements as **user stories**, short statements written from the user's point of view:

```
As a player, I want to choose the board size, so that I can play a longer
```

The format is *As a (who), I want (what), so that (why)*. The "so that" part matters, because the

reason often shows a simpler way to meet the need. Each story is followed by its acceptance criteria. Stories keep the focus on the person who benefits, instead of on technical details.

Priorities and scope

You can almost never build everything, so decide what matters most. A common scheme is **MoSCoW**:

Level	Meaning
Must	The system fails without it
Should	Important, but there is a workaround
Could	Nice to have if time allows
Won't (this time)	Agreed *not* to be built now

The last row is the most useful. Writing down what is **out of scope** prevents the slow growth of a project through "one more small thing," known as scope creep. It also gives you something to point to when someone asks for a surprise feature late in the week.

Assumptions and open questions

Whenever you do not know something, you make an assumption, whether you notice or not. Write each one down: "Assumed: a player who runs out of time loses the turn, not the game." Then mark it for confirmation. Flag anything ambiguous and get clarification before moving to design. An assumption made here often becomes a costly mistake later.

Requirements change

Requirements are not frozen. Clients learn what they want as they see the system, and the world changes. That is normal. The goal is not to prevent change but to make it visible: record changes, note the date, and check what else they affect. A short written list that everyone can see beats a different version in everyone's head.

A Worked Example: Tic-Tac-Toe Requirements

Here is the beginning of a requirements list for the Tic-Tac-Toe project. Each requirement gets an

ID, a type, a priority, and a way to verify it:

ID	Requirement	Type	Priority	How verified
R1	Two marks, X and O. X always moves first.	Functional	Must	Test that the first mark placed is X
R2	A move is a row and column numbered from 1. Occupied, off-board, or non-numeric moves are rejected without using the turn.	Functional	Must	Tests for taken, off-board, and bad input
R3	The game detects a win (across, down, or diagonal) and a draw.	Functional	Must	Tests for each win direction and a full board
R4	The player picks a 3 by 3, 4 by 4, or 5 by 5 board. The win length is 3, 4, and 4.	Business rule	Must	Tests for each size and win length
R5	The game has a two-player mode and a mode against the computer.	Functional	Must	Manual play of both modes
R6	The computer has Easy (random), Medium (win, then block, then random), and Hard (minimax) levels.	Functional	Must	AI tests per level
R7	The Hard AI never loses on a 3 by 3 board.	Functional	Must	Many games against a random opponent

R8	The Hard AI chooses a move in no more than a few seconds on larger boards.	Non-functional	Should	Time a move on a 5 by 5 board
R9	Moves can be timed at 15 seconds. A late move forfeits the turn. The computer is never timed.	Functional	Should	Fake clock tests
R10	Game logic never calls `input()` or `print()`. Only the menu file does.	Non-functional (maintainability)	Must	Code review
R11	Tests never wait for real time or depend on real randomness.	Non-functional	Must	Test suite review
R12	Runs in the terminal on Python 3.10 or later, using the standard library (plus pytest for tests).	Constraint	Must	Run on a clean setup
R13	Tests and a style check run automatically on every pull request, and the checks pass before merging.	Non-functional	Should	Green checks on the merged pull requests

Out of scope for now: a graphical interface, saving games, playing over a network.

Assumptions to confirm:

- A late move forfeits the *turn*, not the game.

- A mark placed on an occupied square is always rejected, never overwritten.

Not every row needs to be this long on a small project. But writing even five or six of them before coding will change the questions you ask.

System Design Basics

Once you know what to build, system design is where you figure out *how* to build it: the structure, components, and how they interact, before implementation begins.

Design happens at two levels:

- **High-level design** decides the big pieces and how they connect. (Which components exist? What talks to what?)
- **Low-level design** decides the details inside a piece. (What classes and functions? What are their inputs and outputs?)

Start high, then go low. Do not worry about function names until you know which pieces exist.

1. The data model

What does the information in your system look like as data structures? For each thing, decide its fields and the type of each.

Thing	Fields (and types)
Board	size (whole number), grid (list of lists of marks), win length (whole number)
Mark	`"X"`, `"O"`, or empty (a one-character string)
Move	row and column (a tuple of two whole numbers)
Settings	board size, mode (two-player or computer), AI level, timed or not

Also decide the **relationships**. A game *has a* board. A game *has an* AI player in computer mode. An AI player *is a* kind of player, and there are three kinds. These are the "has-a" and "is-a" relationships from the OOP principles article.

Data model decisions matter because they are hard to change later. If you store the board as a list of lists, every part of the code that reads it depends on that choice. Wrapping it in a `Board` class, as you did in the classes article, limits the damage if you change your mind.

2. Components

What are the distinct parts, and what is each one responsible for? Even a small app separates into layers:

Layer	Responsibility	Tic-Tac-Toe
Interface	Talks to the user: menus, prompts, printing	`main.py`
Logic	The rules and decisions, with no input or output	`board.py` , `ai.py` , `game.py`
Storage	Keeps data between runs	None in this project (a file or database in later weeks)

Keeping the logic separate from the interface is the single most useful design rule you have learned. It is what makes the logic testable, and what lets you swap a terminal menu for a web page later without touching the rules. Give each component one clear job. If you cannot describe a component in one sentence without "and," split it.

3. Interactions

How do the components work together? Trace one complete scenario from start to finish. Here is a single turn in a game against the computer:

```
1. main.py asks the player for a move and reads it.
2. main.py converts the text to a (row, col) pair and passes it to Game.
3. Game checks the move with Board.place_mark. If rejected, Game tells mai
4. If accepted, Game asks Board whether this move won or filled the board.
5. If the game is not over, Game asks the AI for its move with choose_move
6. Game places the AI's mark on the Board and checks again for a win or a
7. Game returns the new state to main.py, which prints the board.
```

You can also draw this as boxes and arrows. A plain-text sketch is enough:

```
    main.py  (menus, input, printing)
       |
       v
     Game  (turns, timer, win and draw checks)
     /    \
    v      v
 Board   AIPlayer  (EasyAI, MediumAI, HardAI)
```

You do not need formal diagrams or heavyweight documentation at this stage. Paper, a whiteboard, or a text file all work. If you want diagrams inside a README, GitHub can draw them from a text format called Mermaid, but that is optional.

4. Interfaces: promises between components

Before you write a component, decide what it promises to the others. The method name, what it takes in, and what it returns form its **interface**, a contract the other components can rely on. For example:

```
Board.place_mark(row, col, mark)  ->  True if placed, False if not allowed
Board.empty_cells()               ->  list of (row, col) pairs
AIPlayer.choose_move(board, mark) ->  a (row, col) pair for an empty cell
MoveTimer.is_expired()            ->  True if the move took too long
```

When the contracts are settled, two people can work on different components at the same time, and each can test theirs alone.

5. Edge cases and error handling

Decide in advance what happens in the awkward situations. Do not leave them to be discovered:

- What if the player types letters instead of numbers?
- What if the board is full and nobody has won?
- What if the AI is asked to move on a full board?

- What if the clock runs out right as the move is entered?

If a requirement does not say what should happen, that is an open question, not an invitation to guess.

6. Trade-offs and a decision log

Most design questions have more than one reasonable answer. Write down what you chose, what you considered, and why. A few lines per decision is enough:

Decision	Options	Chosen	Reason
How to store the board	List of lists, or a dictionary keyed by position	List of lists inside a `Board` class	Matches what you know, and the class hides it
When to check the timer	While the player types, or after they press Enter	After the move is entered	`input()` cannot be interrupted
How the Hard AI searches larger boards	Full search, or limited depth	Limited depth	A full search is too slow
How to share AI behavior	One class with `if` checks, or a base class with children	Base class with children	Polymorphism avoids long `if` chains

Six months later, when someone asks "why did we do it this way?", the log answers. It also stops the team from arguing about the same choice twice.

7. Dry-run the design

Before coding, walk through a few scenarios on paper against your design and look for gaps:

- A normal game from start to a win.
- A draw.
- A forfeited move.
- An invalid entry.

- A game against each AI level.

Ask at each step, "Which component handles this, and do I have a place for it?" Every gap you find now costs minutes. The same gap found halfway through the code costs hours.

Don't over-design

The goal is to think through the structure before you write code, so that you are not redesigning mid-implementation because you missed a basic case. It is not to produce a perfect document. A good design for a project this size fits on one page. Signs of over-designing: diagrams nobody uses, classes for things that need only a function, and plans for features no one has asked for. Design what is needed, build it, and learn from it. You can always adjust.

From Requirements to Design to Tests

The three line up. Each requirement should lead to a part of the design and a way of checking it. A short table shows the chain, which is called **traceability**:

Requirement	Design element	Test
R2: bad moves rejected without using the turn	`Board.place_mark` returns `False`; `Game` does not switch players on `False`	Parametrized off-board test; taken-cell test
R7: Hard AI never loses on 3 by 3	`HardAI.choose_move` with minimax	Many games against a seeded random opponent
R9: late move forfeits the turn	`MoveTimer` with an injected clock	Fake clock tests: within, over, and exactly at the limit

If you cannot fill in the right-hand columns for a requirement, the requirement may be too vague. If a part of your design traces back to no requirement, you may be building something nobody asked for.

A One-Page Plan Template

Here is a template you can copy into a text file or `README` at the start of any project. Filling it in takes under an hour for a project this size:

```
PROJECT PLAN

Goal: (one or two sentences)

Users: (who uses it, and what do they need)

In scope:
-
Out of scope:
-

Requirements:
ID | Requirement | Type | Priority | How verified

Assumptions and open questions:
-

Data model:
(things, their fields, how they relate)

Components:
(name | responsibility | file)

How one scenario flows:
(numbered steps)

Interfaces:
(method | takes | returns)

Decisions:
(decision | options | chosen | reason)

Dry run:
(scenario | gap found | how it was closed)
```

Common Mistakes

Mistake	What happens	Fix
Starting to code before writing	You build the wrong thing, or	Write at least a short list first

requirements	rebuild it	
Vague requirements ("easy," "fast," "smart")	Nobody can tell if they are met	Replace them with numbers or examples
Mixing "what" and "how"	A requirement says "use a list of lists"	Requirements describe the need. Design chooses the method.
Forgetting non-functional requirements	The system works but is too slow or hard to maintain	Ask about speed, safety, and upkeep too
Silent assumptions	A guess turns into a bug	Write each assumption down and confirm it
No out-of-scope list	The project keeps growing	Say what you are not building
Treating requirements as frozen	You hide changes or resist them	Record changes and check what they affect
One giant component	Hard to test and change	Split by responsibility
Logic mixed with input and output	Cannot be tested without typing	Keep a thin interface layer
Over-designing	Time spent on documents nobody reads	Keep the design to the size of the problem
Never dry-running the design	Gaps appear mid-coding	Walk through scenarios on paper first
No record of decisions	The same debates repeat	Keep a short decision log

Try It Yourself

1. Take this request: "I want an app to track my workouts." Write ten clarifying questions you would ask.
2. Rewrite each vague requirement as a testable one: "The app should load quickly." "The form should be user-friendly." "The system should handle errors." "Reports should be accurate."
3. Write three functional and two non-functional requirements for a bookstore checkout that sells physical books and eBooks (the books from the OOP articles).

4. Write two acceptance criteria in Given, When, Then form for the Tic-Tac-Toe rule "a player cannot place a mark on an occupied square."

5. Write a user story, with acceptance criteria, for the timed moves feature.

6. Classify these as functional, non-functional, constraint, or business rule: "The game runs on Python 3.10," "A draw is declared when the board is full," "A move takes under 3 seconds," "The player can choose a board size."

7. Prioritize the Tic-Tac-Toe requirements with MoSCoW. Which would you cut first if you ran out of time, and why?

8. Start the one-page plan template for Tic-Tac-Toe. This is also Part 2 of the Tic-Tac-Toe lab, where you save it as `PLAN.md` with at least 10 requirements, the components, the interfaces, a decision log, and a dry run. Work on it before you write any code.

9. Sketch the component diagram for Tic-Tac-Toe on paper. Add a line for the timer. Where does it belong?

10. Write the interface (method name, inputs, outputs) for each public method of `Board`, `AIPlayer`, and `Game` before writing any code.

11. Add three entries to a decision log for your own project.

12. Dry-run your design with a draw, a forfeited move, and an invalid entry. Write down any gap you find and how you would close it.

13. Build a traceability table for five of your requirements, linking each to a design element and a test.

14. Look at a recent app you use and write down five requirements you think its builders must have started from.

Why This Matters for an FDE

In client-facing work, you are often the one gathering requirements directly from a client who may not describe their needs in precise, technical terms. Turning a vague request ("I just want to see where my money's going each month") into a concrete set of testable requirements is a skill in itself, and you will rely on it constantly in this role. Being able to sketch a basic system design quickly, without over-engineering it, helps you move from conversation to working code faster. It

also protects you: a written, agreed requirements list is what lets you say "that is a new request" instead of absorbing endless unplanned work. This is also where AI-assisted development depends most on you. An AI assistant builds exactly what you describe, so a vague description gets you confident, working code for the wrong problem.

Comprehension Check

1. What is the difference between a functional and a non-functional requirement? Give one example of each for a system you use regularly (for example, a banking app).
2. Why is it risky to start implementation before requirements are clearly defined?
3. Rewrite "The game should respond quickly" as a testable requirement.
4. What does an acceptance criterion in Given, When, Then form look like, and how does it connect to a test?
5. What is the purpose of an "out of scope" list?
6. In the Tic-Tac-Toe design, which layer does `main.py` belong to, and why must the logic layer avoid `input()` and `print()`?
7. For Tic-Tac-Toe, what is one data model decision you would need to make about how the board is stored, and why does it matter?
8. What is an interface in a design, and why decide it before writing the component?
9. What is the point of a decision log? Give one entry for the timer.
10. What does it mean to dry-run a design, and what do you hope to find?
11. A requirement in your list has no matching test. What might that tell you?

← Previous: TDD with pytest

Next: Git & GitHub: Fundamentals, Branching Strategies, and Pull Requests →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!